Week 6: Cross-Site Scripting

Learning Objectives

By the end of this lecture, the student should be able to:

- *Define Cross-Site Scripting and explain why it violates the separation between code and data introduced in Week 1.*
- *Distinguish reflected, stored, and DOM based Cross-Site Scripting.*
- *Explain what an attacker gains by executing a script in a victim's browser, including session theft.*
- *Apply context aware output encoding, selecting the correct encoding rule for the HTML, attribute, JavaScript, CSS, and URL contexts.*
- *Identify the sources and sinks involved in DOM based Cross-Site Scripting when reviewing client-side JavaScript.*
- *Explain how input validation, Content Security Policy, and HttpOnly cookies contribute additional layers of defense.*
- *Describe how modern front-end frameworks such as React and Angular apply automatic escaping by default, and recognize the constructs that bypass this protection.*
- *Locate Cross-Site Scripting within the OWASP Top Ten and explain why the 2021 edition merged it into the Injection category.*

Definition of Cross-Site Scripting

Cross-Site Scripting, commonly abbreviated XSS, occurs when an application includes untrusted input in a page without properly neutralizing it, allowing that input to be interpreted by the browser as executable script rather than as plain text. The browser has no way of distinguishing a legitimate script the developer intended to send from a malicious script an attacker managed to insert; both arrive from the same trusted origin and both are executed with the same privileges. The underlying flaw is identical to the one examined in Week 1 for SQL Injection: user input crosses into a context where it is interpreted as instructions instead of as data, this time inside the browser's HTML and JavaScript engine rather than inside a database query.

Three Categories of Cross-Site Scripting

Cross-Site Scripting is classified according to where the malicious payload lives and how it reaches the victim.

Category	Where the payload lives	Persistence	Typical delivery
Reflected	Included in the request itself, such as a URL parameter	Not stored; exists only for a single response	A crafted link sent to the victim, often through phishing
Stored	Saved on the server, such as in a comment or a profile field	Persistent; served to every visitor of the affected page	No further interaction needed once the payload is posted
DOM based	Never reaches the server; written into the page by vulnerable client-side script	Depends on the vulnerable script, often not stored	Manipulating a URL fragment or another client-side input source

Sources and Sinks in DOM Based Cross-Site Scripting

DOM based Cross-Site Scripting deserves closer attention because it never touches the server at all, so server-side defenses such as output encoding on the backend cannot reach it. Reviewing client-side JavaScript for this category requires two standard terms borrowed directly from static analysis. A source is any point where untrusted data enters the script, most commonly a piece of the current URL or a value handed to the page by the browser itself. A sink is any function or property that can turn a plain string into markup that is parsed or code that is executed. A vulnerability exists precisely when a value flows from a source into a sink without passing through a safe transformation in between.

Category	Representative examples
Sources (untrusted input reaching client-side script)	***location.hash***, ***location.search***, ***document.referrer***, ***document.URL***, ***window.name***, and data delivered through ***postMessage***.
Sinks (constructs that turn a string into executed code or rendered markup)	***element.innerHTML*** and ***element.outerHTML***, ***document.write***() and ***document.writeln(), eval(), new Function(), setTimeout()*** or ***setInterval()*** when given a string argument, and an event handler attribute set directly from untrusted data.

The safest response to this category is to avoid the dangerous sinks entirely wherever possible, favoring ***element.textContent***, which always treats its argument as displayable text, or building elements explicitly through the document object model rather than assembling markup as a string.

A Detailed Walkthrough of Stored Cross-Site Scripting

Stored Cross-Site Scripting is the most consequential of the three categories, since a single successful injection can affect every subsequent visitor without any further action from the attacker.

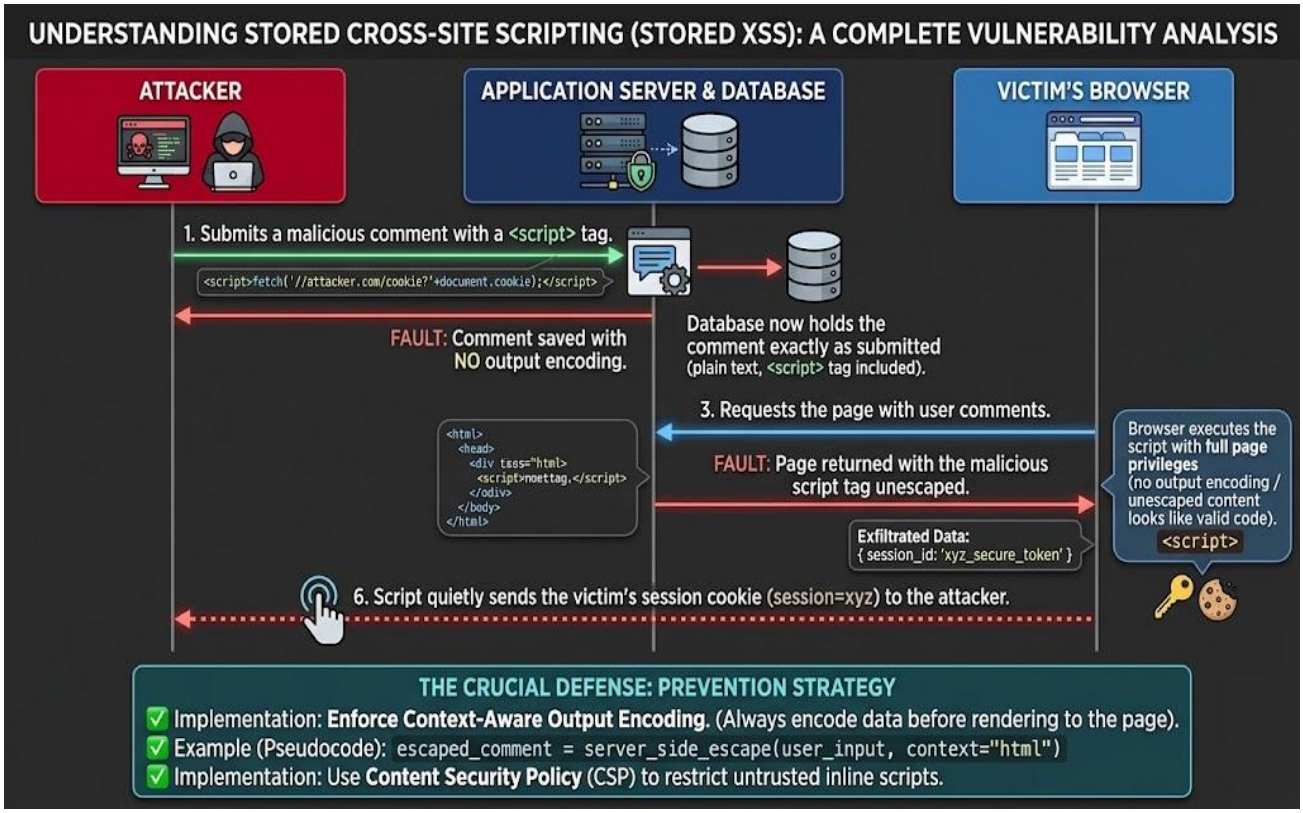


Figure 1. A single malicious comment, once stored without encoding, executes inside the browser of every visitor who later views the page.

The comment field in this example behaves in the same way as the login field examined in Week 1. Both accept text from an untrusted source, and both fail only at the moment that text is treated as something other than plain data.

What an Attacker Gains

A script running inside a victim's browser, under the victim's own authenticated session, can perform almost anything the victim could do manually. The most common objective is session theft: reading the session cookie established in Week 3 and sending it to a server the attacker controls, after which the attacker can impersonate the victim without ever knowing a password. Beyond cookie theft, the same script can capture keystrokes, redirect the page to a convincing fake login form, silently submit requests on the victim's behalf, or simply deface the page. Several of these outcomes, particularly forcing an action on the victim's behalf, reappear in Week 7 under Cross-Site Request Forgery.

Output Encoding as the Primary Defense

The core defense against Cross-Site Scripting mirrors the core defense against SQL Injection from Week 1: untrusted input must never be interpreted as code. In the browser, this means applying output encoding at the exact point where data is written into a page, converting characters with special meaning, such as angle brackets, into their harmless text equivalents.

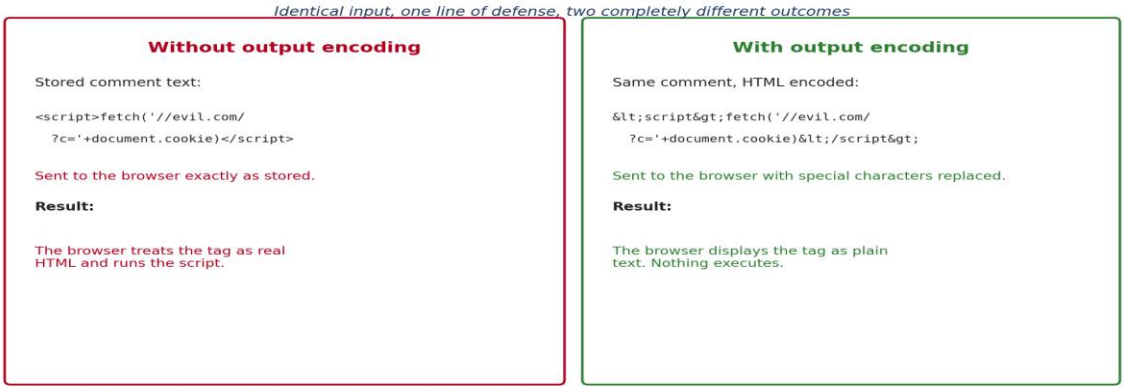


Figure 2. The same submitted text produces two outcomes, entirely dependent on whether the application encoded it before writing it into the page.

Output context	Where it appears	Correct encoding rule
HTML body	Text placed between ordinary tags, such as inside a paragraph or a division	HTML entity encoding of the characters with special meaning, including the angle brackets, the ampersand, and the quotation marks
HTML attribute	A value placed inside an attribute, such as a value or an ***href***	Entity encoding of every character outside a small safe set, since an attribute can be closed by characters other than an angle bracket
JavaScript variable	A value placed inside a script block or an inline event handler, always as an already quoted string	Unicode escaping of every character, combined with never placing the value anywhere in the script other than inside an existing quoted string
CSS value	A value placed inside a stylesheet or a style attribute	CSS hexadecimal encoding, since several CSS constructs can trigger unexpected network requests or behavior
URL parameter	A value placed inside a query string or another portion of a URL	Percent encoding of every reserved and unsafe character

Encoding must match its context. Text placed inside an HTML element requires HTML encoding; text placed inside a JavaScript block or an HTML attribute requires a different encoding scheme entirely. Applying the wrong encoding for the surrounding context leaves the application just as exposed as applying none. Five contexts account for nearly every case a developer encounters in practice.

One context deserves a separate warning. Placing untrusted data directly inside a script tag, rather than inside an already quoted string within that script, is considered a dangerous context in its own right. No encoding scheme fully neutralizes this placement, since the browser's JavaScript parser interprets the surrounding structure before any encoded characters are read back as literal text. The only reliable approach is to avoid this placement altogether and pass data into scripts through a quoted variable instead.

Additional Layers of Defense

Output encoding is necessary but should not stand alone. Input validation restricts what a field accepts in the first place, ideally using an allow list of permitted characters or formats rather than attempting to block known malicious patterns. Where an application must accept some HTML, such as a rich text comment field, a dedicated sanitization library should parse and rebuild the markup rather than relying on custom filtering code. The Content Security Policy header, introduced in Week 2, adds a further layer that limits the damage even when an injection slips past encoding, since a strict policy can prevent inline scripts from executing and restrict which external domains a page may load script from at all. Finally, the ***HttpOnly*** cookie attribute, also introduced in Week 2, does not prevent Cross-Site Scripting from occurring, but it does prevent an injected script from directly reading the session cookie, closing off the most damaging single consequence of a successful attack.

Automatic Escaping in Modern Front-End Frameworks

Most applications built today do not assemble HTML by hand; they use a front-end framework such as React or Angular, and both apply automatic escaping as their default behavior. In React, a value placed inside curly braces within markup is converted to a plain string and escaped before it is written to the page, so ordinary use of the framework already applies HTML body encoding without the developer requesting it. React exposes a deliberately named escape hatch, ***dangerouslySetInnerHTML***, for the rare case where raw markup must be rendered; the unusual name is intentional, and the framework's own documentation warns against passing user supplied content through it without an independent sanitization library. Angular follows the same philosophy through its ***DomSanitizer*** service, which automatically sanitizes values bound into HTML, style, and URL contexts, stripping dangerous constructs such as a script

element while preserving safe content. Angular likewise exposes explicit bypass methods, including ***bypassSecurityTrustHtml,*** for the case where a value has already been verified safe through other means.

Neither framework eliminates XSS as a possibility. Each framework simply makes the safe behavior the default rather than something the developer must remember to apply on every output. The moment a developer reaches for an escape hatch such as ***dangerouslySetInnerHTML*** or ***bypassSecurityTrustHtml***, responsibility shifts back onto that developer, and the same context aware encoding discipline described earlier in this lecture applies again in full.

Placement Within the OWASP Top Ten

In earlier editions of the OWASP Top Ten, Cross-Site Scripting stood as its own category. The 2021 edition, used as the reference throughout this course, folded it into A03:2021, Injection, alongside SQL Injection and several related weaknesses. The reasoning is direct: both vulnerabilities share the same root cause, untrusted input reaching an interpreter that cannot distinguish code from data, and differ only in which interpreter is involved, the database engine in one case and the browser's rendering engine in the other.